

Konstrukturen und Garbage Collection – Teil 1

Stack, Heap und Objekterstellung

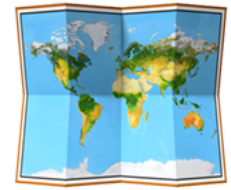
Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 5. Januar 2026 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstrukturen und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Konstrukturen und Garbage Collection

 Plan für diese Woche

 Stack und Heap

 Konstrukturen

 Objektinitialisierung

 Überladene Konstrukturen I

 Superklassenkonstrukturen

 Überladene
Konstrukturen II

 Pair Programming

 Garbage Collection

 Spaß

📖 Konstruktoren und Garbage Collection

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

📚 Was „lebt“ auf dem Stack und was „lebt“ auf dem Heap?

🏗️ Wie und wozu wird ein Konstruktor aufgerufen?

🚧 Welche Arten von Konstruktoren werden unterschieden?

🧱 Was kann von überladenen Methoden auf Konstruktoren übertragen werden?

👤 Welche Fragen haben Sie zu diesem Kapitel?

9 Konstruktoren und Garbage Collection

Leben und Sterben eines Objekts



Objekte werden geboren und Objekte sterben. Sie als Programmierer/-in sind für den Lebenszyklus eines Objekts verantwortlich. Sie entscheiden, wie es **konstruiert** wird. Sie entscheiden, wenn es **zerstört** wird. Nur, dass Sie das Objekt nicht selbst *zerstören*, Sie *geben* es einfach *auf*. Aber sobald Sie es aufgegeben haben, kann der herzlose **Garbage Collector (gc)** es pulverisieren und den Speicher, den das Objekt belegte, wieder anderweitig nutzen. Wenn Sie Java schreiben, werden Sie Objekte erstellen. Früher oder später müssen Sie einige von ihnen gehen lassen, oder Sie riskieren, dass der Arbeitsspeicher knapp wird. In diesem Kapitel sehen wir uns an, wie Objekte erstellt werden, wo sie leben, während sie leben, und wie man sie effizient gehen lässt. Das heißt, wir sprechen über den Heap, den Stack, den Geltungsbereich (Scope), Konstruktoren, Superklassenkonstruktoren, Nullreferenzen und mehr. Warnung: Dieses Kapitel enthält Material über den Tod von Objekten, den Sie als Lesende möglicherweise beunruhigend finden. Am besten gehen Sie keine emotionale Bindung ein.



Stack und Heap

Arbeitsspeicher

📖 Stack und Heap

- Wenn die JVM gestartet wird, erhält sie vom Betriebssystem einen Teil des Arbeitsspeichers, um das Java-Programm auszuführen.
- Für uns Programmierer:innen sind dort zwei Speicherbereiche wichtig: der **Stack** und der **Heap**.
- Hier „leben“ in Java:
 - Methodenaufrufe und lokale Variablen (Stack)
 - Objekte (Garbage Collectible Heap)

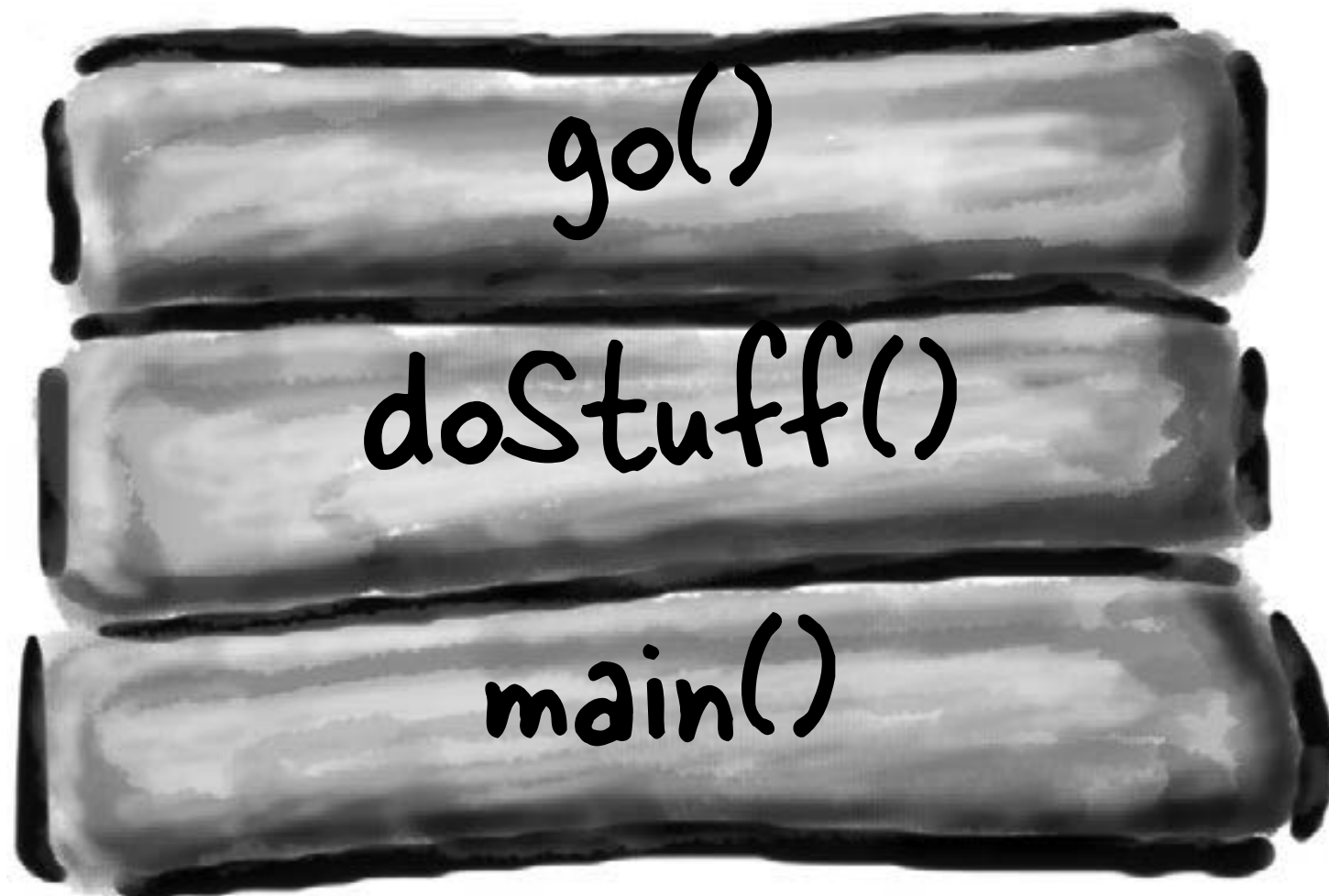


Methodenaufrufe, lokale Variablen und Objekte

Stack und Heap

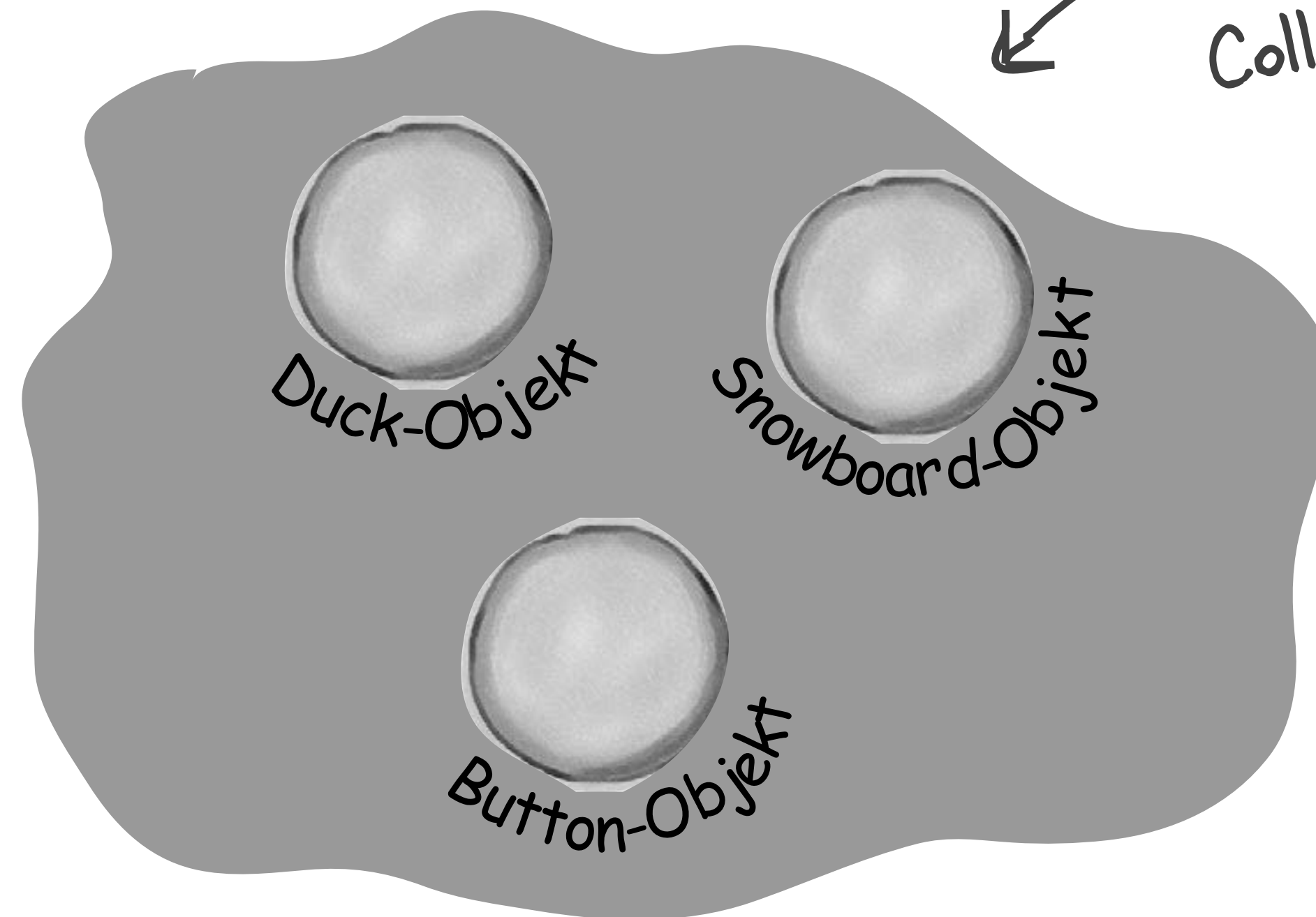
Der Stack

Wo Methodenaufrufe und lokale Variablen wohnen.



Der Heap

Wo **ALLE** Objekte wohnen.



Auch bekannt als
»Der Garbage
Collectible Heap«.

```
class Horse {  
    private double height = 15.2;  
    private String breed;  
    // ...  
}
```

Instanzvariablen werden innerhalb einer Klasse, aber nicht innerhalb einer Methode deklariert.

```
class AddThing {  
    int a;  
    int b = 12;  
  
    public int add() {  
        int total = a + b;  
        return total;  
    }  
}
```

Lokale Variablen werden innerhalb einer Methode deklariert.

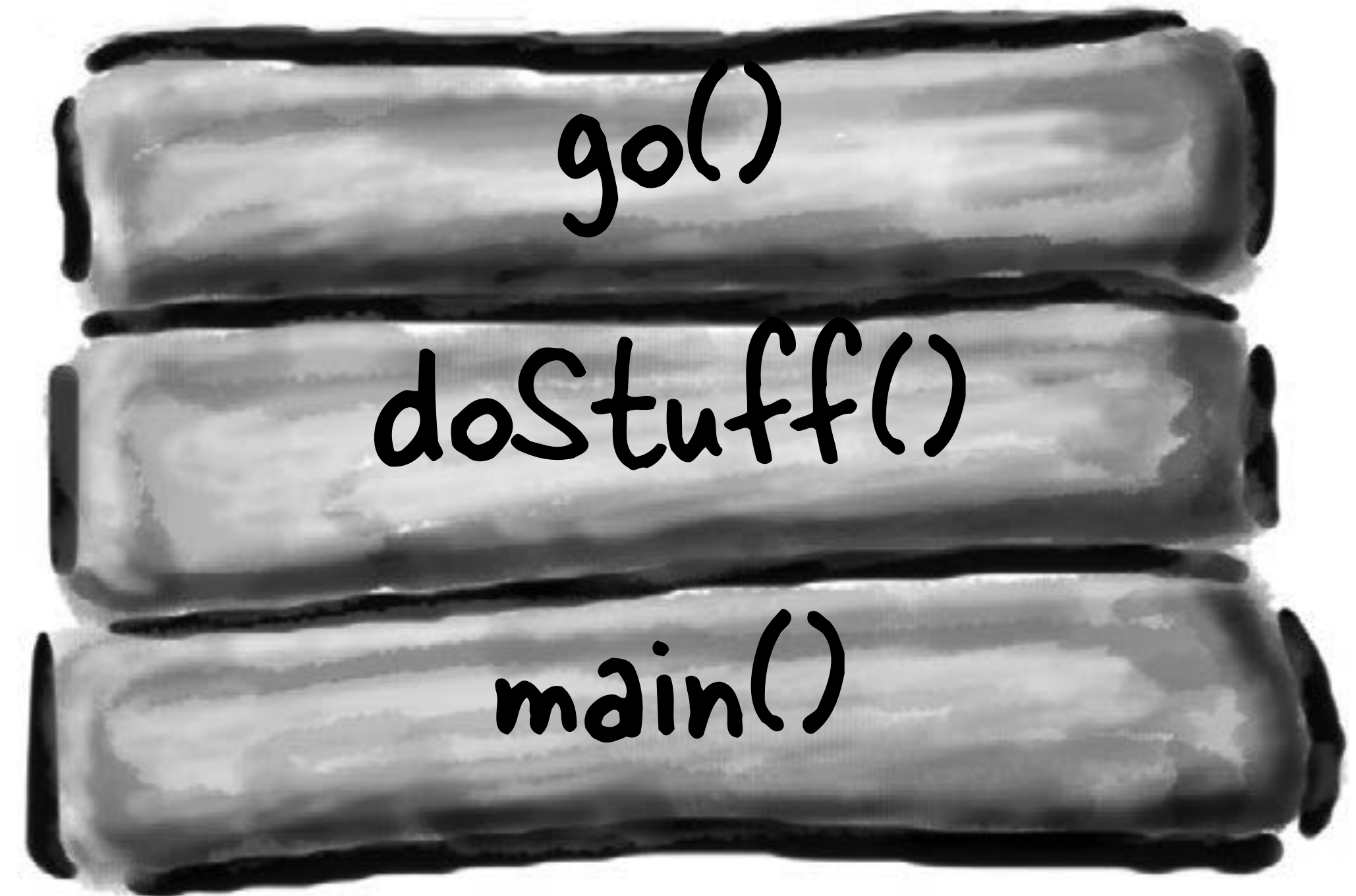
Lokale Variablen

Stack und Heap

- Lokale Variablen werden in einer Methode deklariert...
 - ... und in den Parametern der Methode:
`void add(int a, int b) { ... }`
- Lokale Variablen haben eine sehr begrenzte Lebensdauer.
 - Sie leben so lange, wie sich die Methode auf dem Stack befindet.
 - Solange die Methode nicht die schließende geschweifte Klammer erreicht.

Der Stack

Wo Methodenaufrufe und lokale Variablen wohnen.

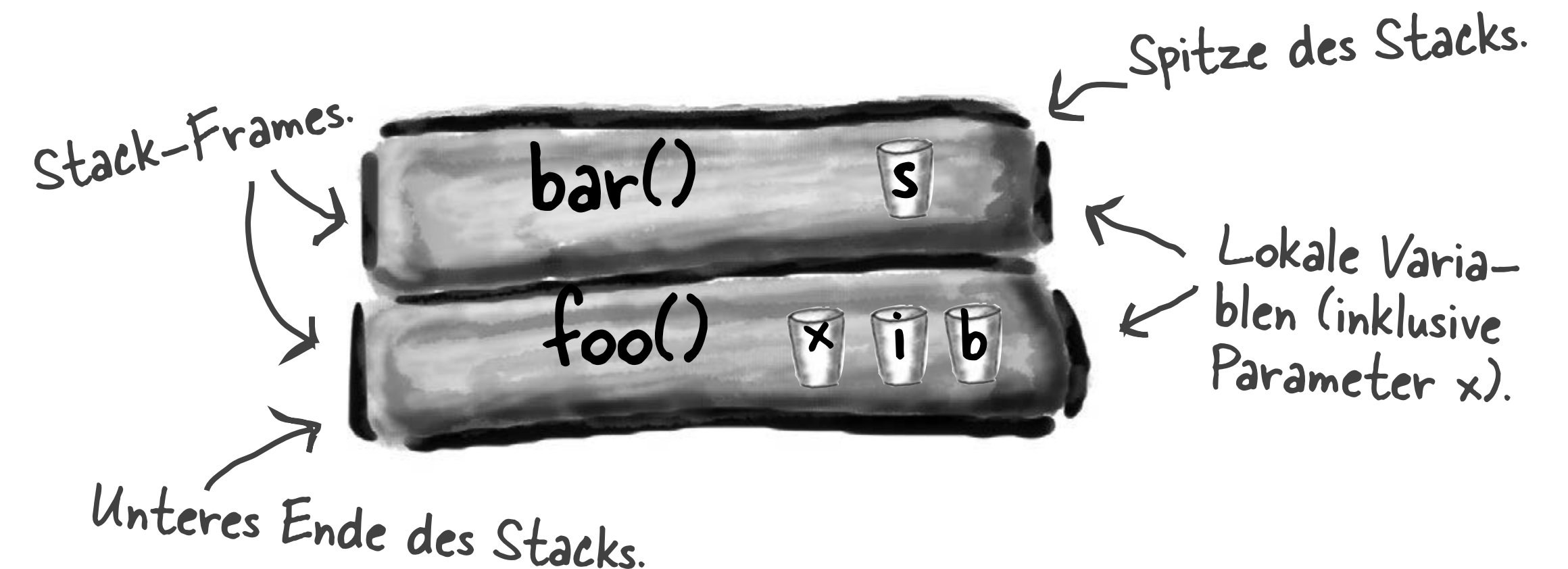


Methodenstapel

Stack und Heap

- Rufen Sie eine Methode auf, landet diese oben auf dem **Call-Stack** (Aufruf-Stapel).
- Dazu wird ein **Stack-Frame** erzeugt, der – vereinfacht gesagt – den Zustand der Methode enthält.
- Bei jedem Methodenaufruf wird ein neuer Stack-Frame erzeugt.
- Die Methode oben auf dem Stack ist immer die Methode, die gerade ausgeführt wird.
- Wenn eine Methode ihre schließende geschweifte Klammer erreicht, „verschwindet“ sie vom Stack.

Ein Call-Stack mit zwei Methoden



Ein Stack-Szenario

Stack und Heap

```
public void doStuff() {  
    boolean b = true;  
    go(4);  
}
```

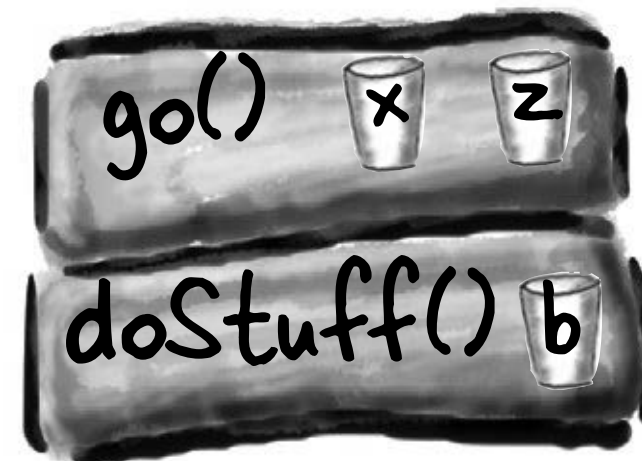
```
public void go(int x) {  
    int z = x + 24;  
    crazy();  
    // mehr Code ...  
}
```

```
public void crazy() {  
    char c = 'a';  
}
```

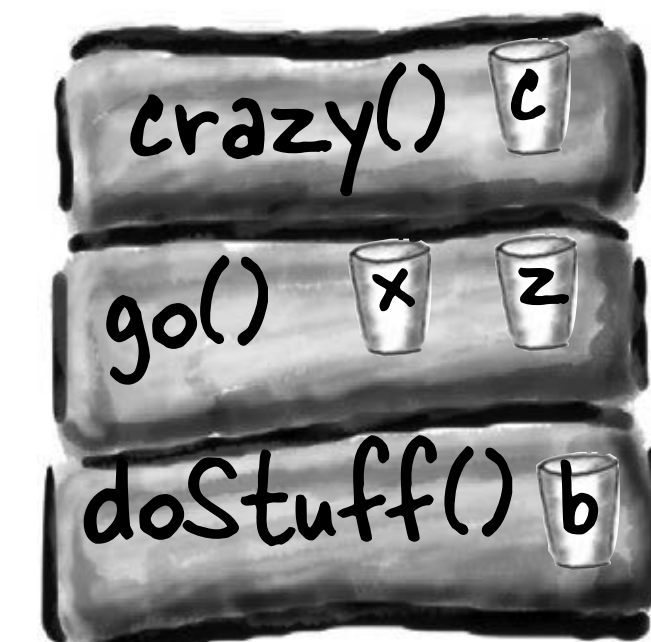
① Code aus einer anderen Klasse ruft **doStuff()** auf, und **doStuff()** landet im Stack-Frame an der Spitze des Stacks. Die boolesche Variable namens »b« landet im Stack-Frame von **doStuff()**.



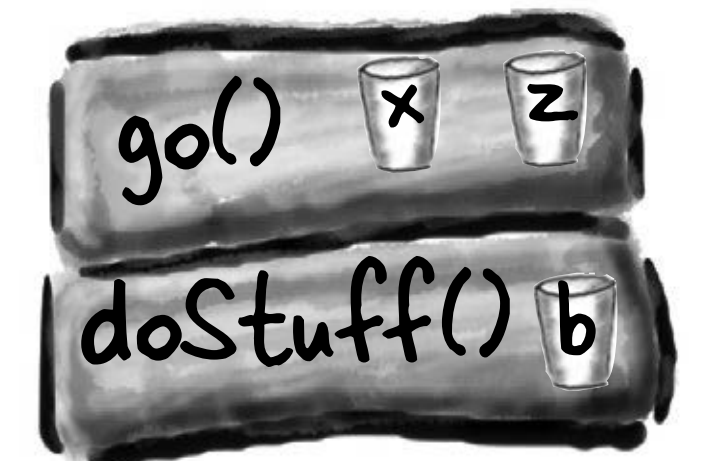
② **doStuff()** ruft **go()** auf, und **go()** wird an die Spitze des Stacks geschoben. Die Variablen »x« und »z« befinden sich im Stack-Frame von **go()**.



③ **go()** ruft **crazy()** auf. Jetzt befindet sich **crazy()** an der Spitze des Stacks mit der Variablen »c« im entsprechenden Frame.



④ **crazy()** wird beendet, und ihr Stack-Frame wird vom Stack entfernt. Die Ausführung geht zurück zur Methode **go()** und macht auf der Zeile weiter, die auf den Aufruf von **crazy()** folgt.



„Wozu lernen wir den ganzen Stack- und Heap-Kram? Ich schreibe lieber direkt Java-Code...“

Ulli – ungeduldige Student:in im 1. Semester



„Dies ist wichtig, um den Gültigkeitsbereich von Variablen (Scope), Probleme bei der Objekterzeugung, Speicherverwaltung, Threads und Exception-Handling zu verstehen.“

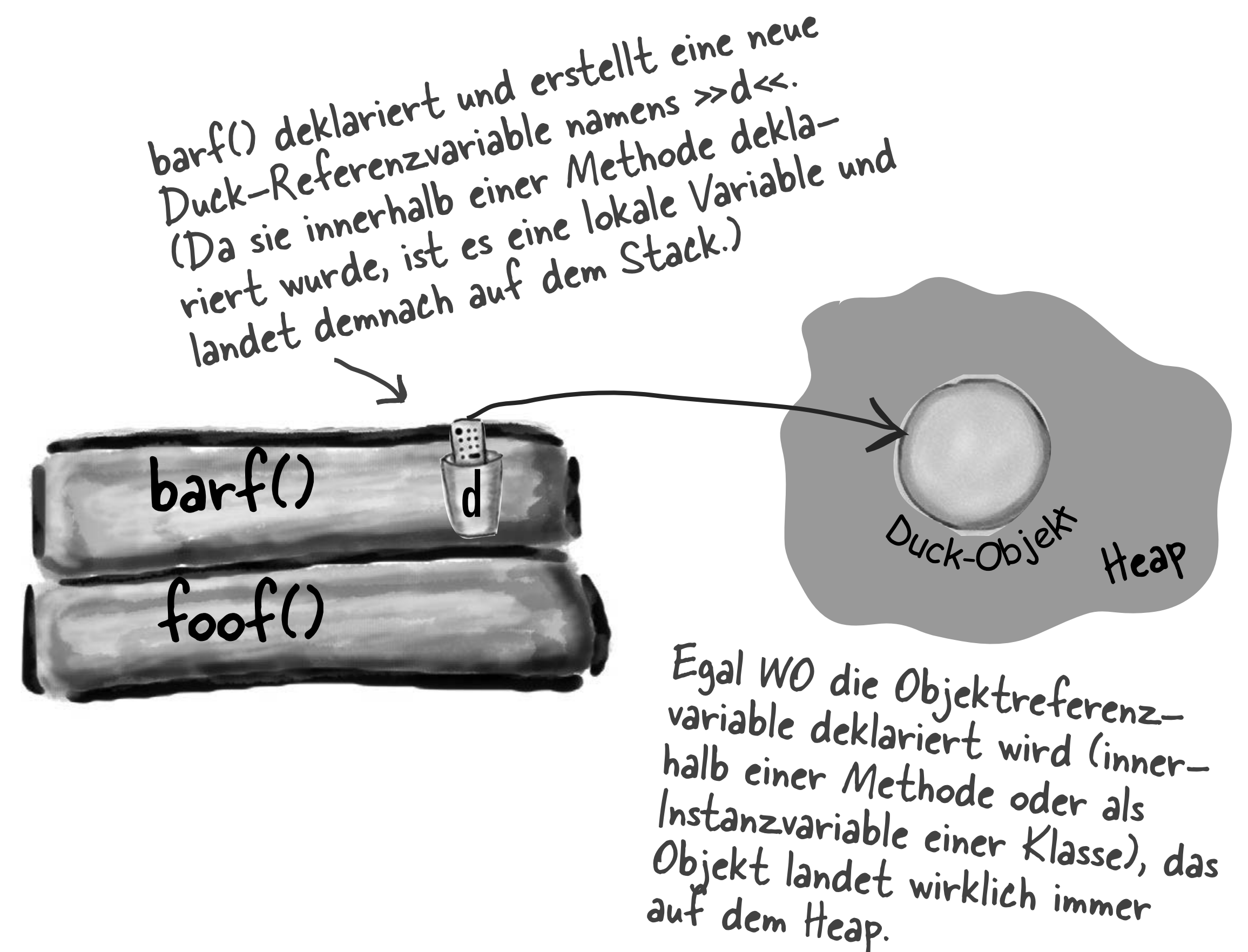
Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt



Lokale Objekte

📖 Stack und Heap

- Eine nicht elementare Variable enthält eine **Referenz** auf ein Objekt – nicht das Objekt selbst.
- Objekte befinden sich immer auf dem Heap. Unabhängig davon, ob sie innerhalb einer Methode oder als Instanzvariable deklariert wurden.
- Ist eine lokale Variable eine Referenz auf ein Objekt, landet nur die Variable, also die Referenz oder „Fernbedienung“, auf dem Stack.



Jetzt Seite 241 wiederholen!



Sie haben dafür 3 Minuten Zeit.

! ? Quiz

📖 Stack und Heap

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



In Java gibt es zwei Speicherbereiche, die für uns von Bedeutung sind: den Stack und den Heap.

Richtig!

Instanzvariablen werden innerhalb einer Klasse und innerhalb von Methoden deklariert.

... aber außerhalb von Methoden deklariert.

Lokale Variablen sind Variablen innerhalb von Methoden oder Parametern von Methoden.

Richtig!

Alle lokalen Variablen leben auf dem Stack in dem Frame, der zu der Methode gehört, in der die Variablen deklariert werden.

Richtig!

Objektreferenzvariablen funktionieren genau wie elementare Variablen - wenn die Referenz als lokale Variable deklariert wird, landet sie auf dem Heap.

... auf dem Stack.

Alle Objekte leben auf dem Heap,
unabhängig davon, ob die Referenz eine
lokale oder eine Instanzvariable ist.

Richtig!

Instanzvariablen leben auf dem Stack
innerhalb des Objekts, zu dem sie
gehören.

... auf dem Heap...

Wenn die Instanzvariable eine Referenz auf ein Objekt ist, befinden sich die Referenz und das Objekt, auf das sie sich bezieht, auf dem Heap.

Richtig!



Welcher **eine** Gedanke zu
„ Stack und Heap“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Konstrukturen



Ein Objekt erstellen

 Objektreferenzen

```
Dog myDog = new Dog();
```



Deklaration, **Erstellung** und Zuweisung von Objekten

- Weist die JVM an, Speicherplatz auf dem Heap für ein neues Dog-Objekt zuzuweisen.
- Dies werden wir im Kapitel „Konstruktoren und Garbage Collection“ genauer betrachten.

Konstruktor-Aufruf

Konstrukturen

- Bei `new Dog()` handelt es sich um den **Aufruf eines Konstruktors**.
- Ein Konstruktor sieht einer Methode sehr ähnlich – ist aber keine.
- Ein Konstruktor enthält den Code, der ausgeführt wird, wenn Sie **new** auf einem Klassentyp aufrufen.
- Die JVM findet diese Klasse und ruft den Konstruktor auf.



„Ich hab’ noch nie einen Konstruktor in meinen Klassen deklariert, also sollte es auch keinen geben, oder?“

Merle – aufmerksame Student:in im 1. Semester





„Genau, und wichtig: Dieser sogenannte Standardkonstruktor ruft einfach den Konstruktor der Oberklasse ohne Argumente auf...”

„Doch! Wenn du keinen selbst schreibst, erzeugt der Compiler einen Konstruktor für uns: `public Dog() { }`“

Gerit – aufgeweckte Student:in im 1. Semester



```
1 public class ConstructionDemo {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Using the new operator to create new Dog...");  
5  
6         // 1) Instantiation: The *new* keyword is a Java operator that creates the object.  
7         Dog d = new Dog(); // 4) It is only at this point...  
8         // ... that the new object is assigned to the variable.  
9  
10        System.out.println("New anonymous doggie: " + d);  
11    }  
12  
13 }  
14
```

STDIN

Input for the program (Optional)

Output:

Using the new operator to create new Dog...
Woof! I'm a 'new-born' puppy.
New anonymous doggie: Dog@7ad041f3

! ? Quiz

🏗️ Konstruktoren

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein Konstruktor ist der Code, der ausgeführt wird, wenn auf einem Klassentyp `new` aufgerufen wird.

Richtig!

Ein Konstruktor muss den gleichen Namen wie die Klasse haben und muss genau einen Rückgabebetyp haben.

... darf keinen Rückgabebetyp haben.

Mit Hilfe eines Konstruktors können Sie den Zustand (die Instanzvariablen) des zu konstruierenden Objekts initialisieren.

Richtig!

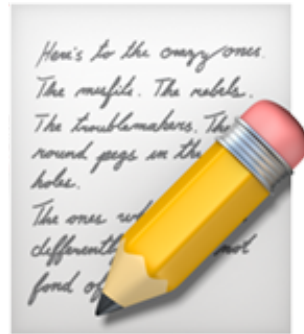


Welcher **eine** Gedanke zu



„Konstruktoren“ war für

Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Objektinitialisierung



```
1 public class InitializationDemo {  
2  
3     public static void main(String[] args) {  
4         Dog anon = new Dog();  
5         System.out.println("New anonymous doggie: " + anon);  
6  
7         Dog luna = new Dog();  
8         luna.setName("Luna");  
9         System.out.println("New doggie known by name: " + luna);  
10  
11         // Wait a minute! *I* have to remember to call the setter?  
12         // This could result in a Dog object that is *not* properly initialized.  
13     }  
14  
15 }  
16
```

STDIN

Input for the program (Optional)

Output:

```
Woof! I'm a 'new-born' puppy.  
New anonymous doggie: Dog[null]  
Woof! I'm a 'new-born' puppy.  
New doggie known by name: Dog[Luna]
```

```
1 public class InitializationDemo {  
2  
3     public static void main(String[] args) {  
4         Dog luna = new Dog("Luna"); // Phew! This way we are on the safe side.  
5         System.out.println("New doggie known by name: " + luna);  
6     }  
7  
8 }  
9 |
```

STDIN

Input for the program (Optional)

Output:

Woof! I'm a 'new-born' puppy...
Woof-woof! ... and I'm called Luna.
New doggie known by name: Dog[Luna]

! Quiz

🚧 Objektinitialisierung

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wenn Sie Ihrer Klasse keinen Konstruktor hinzufügen, fügt der Compiler einen Standardkonstruktor für Sie ein.

Richtig!

Der Standardkonstruktor ist grundsätzlich argumentlos.

Richtig!

Wenn Sie Ihrer Klasse selbst einen Konstruktor hinzufügen, erzeugt der Compiler auch einen Standardkonstruktor.

... keinen Standardkonstruktor.

Wenn Sie einen Konstruktor ohne Argumente benötigen und die Klasse bereits einen Konstruktor mit Argumenten enthält, müssen Sie den Konstruktor ohne Argumente selbst erstellen.

Richtig!

Stellen Sie nach Möglichkeit immer einen Konstruktor ohne Argumente zur Verfügung. Damit erleichtern Sie es anderen Programmierer:innen, ein einsatzfähiges Objekt zu erstellen. Geben Sie dabei sinnvolle Standardwerte an.

Richtig!

 Welcher **eine** Gedanke zu
„ Objektinitialisierung“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Überladene Konstruktoren I

```
1 public class OverloadingDemo {  
2  
3     public static void main(String[] args) {  
4         Dog luna = new Dog("Luna");  
5         System.out.println("New doggie known by name: " + luna);  
6  
7         Dog some = new Dog(null);  
8         System.out.println("Some new doggie: " + some);  
9  
10        // Wait a minute! *I* need to know that I have to pass null?  
11        // That's a bit of a hassle, isn't it?  
12        // And anyway, what if Rocky is really a she-dog?  
13        // We can do better than that!  
14    }  
15  
16 }  
17
```

STDIN

Input for the program (Optional)

Output:

Woof! I'm a 'new-born' puppy...
Woof-woof! ... and I'm called Luna.
New doggie known by name: Dog[Luna]
Woof! I'm a 'new-born' puppy...
Yelp! ... and I'm embarrassingly named Rocky.
Some new doggie: Dog[Rocky]

```
1 public class OverloadingDemo {  
2  
3     public static void main(String[] args) {  
4         Dog luna = new Dog("Luna");  
5         System.out.println("New doggie known by name: " + luna);  
6  
7         Dog some = new Dog(); // Whew! Now that's more like it.  
8         System.out.println("New doggie: " + some);  
9     }  
10  
11 }  
12
```

STDIN

Input for the program (Optional)

Output:

```
Woof! I'm a 'new-born' puppy...  
Woof-woof! ... and I'm called Luna.  
New doggie known by name: Dog[Luna]  
Woof! I'm a 'new-born' puppy...  
Woof-woof! ... and I'm appropriately named Bailey.  
New doggie: Dog[Bailey]
```

„Verstehe, wenn meine Klasse mehr als einen Konstruktor enthält, dann bedeutet das, dass ich **überladene Konstrukturen** habe – ähnlich wie es bei Methoden der Fall ist.“

Merle – aufmerksame Student:in im 1. Semester



**Jetzt Seiten 248
und 249
wiederholen!**



Sie haben dafür 5 Minuten Zeit.

! Quiz

📦 Überladene Konstruktoren I

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Überladene Konstruktoren bedeuten, dass in Ihrer Klasse nur ein Konstruktor vorhanden ist.

... mehr als ein Konstruktor...

Überladene Konstruktoren müssen
verschiedene Argumentlisten besitzen.

Richtig!

Es kann nicht zwei Konstruktoren mit der gleichen Argumentliste geben. Eine Argumentliste enthält die Reihenfolge und die Typen der Argumente.

Richtig!

Instanzvariablen wird ein Standardwert zugewiesen, auch wenn Sie dies nicht explizit tun. Die Standardwerte sind 1/1.0/true für Elementartypen und new Object() für Referenzen.

Die Standardwerte sind 0/0.0/false für Elementartypen und null für Referenzen.

 Welcher **eine** Gedanke zu
„ Überladene Konstruktoren I“ war
für Sie am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Konstruktoren und Garbage Collection

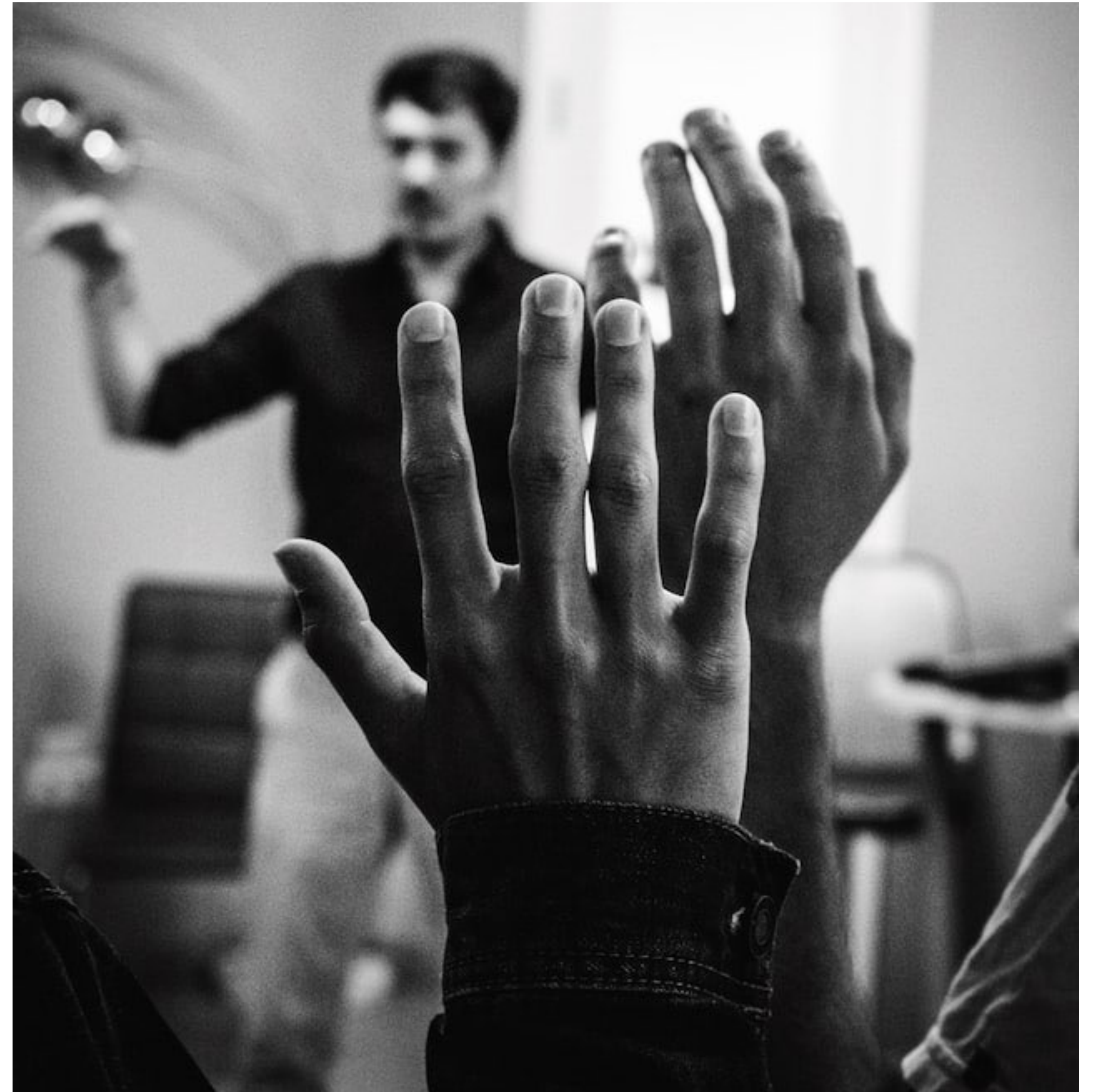
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Konstruktoren und Garbage Collection?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Konstrukturen und Garbage Collection



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

